

# High-level architecture

How one Next.js app and one PostgreSQL database run all of the body’s 80 awards, with every difference between awards held as configuration instead of code.

Drawn from the Awards Platform Product and Technical Specification dated 2 October 2026 (Priyanshu Phulara). Every module, store and rule here traces back to a section of that document. Where the spec is silent or contradicts itself, the drawing says so, and pages 10 and 11 list what is still open.

## The architecture in five decisions

### Modular monolith

One Next.js deployable and one database. Fifteen modules with strict boundaries, so any one could be split out later. At 24,000 to 40,000 applications a year nothing needs separate services.

### An award is data, not code

Questionnaire and scoring sheet are versioned JSON. Code reads behaviour from settings such as `cycle.blindJudging` and `cycle.feePaise` and never names an award.

### Shared tables, every row tied to its cycle

All awards use the same tables. Foreign keys, roles scoped to a department, award or cycle, and the service layer keep one award’s data out of another’s.

### Every check lives in a service

Each service function takes the actor first, checks scope, applies the rule, writes in one transaction with its audit event and returns a view built for that role. Routes and screens stay thin.

### No background jobs

The deadline lock is computed from one clock helper on every write, so nothing runs on a timer. Emails go out while the request is handled; page 10 explains why that needs a second look.

### Load the design is sized for

About 80 awards with 300 to 500 applications each. The largest has 250 indicators in 15 areas: about 125,000 scores in one round. A few hundred applicants are online just before a deadline. One PostgreSQL database handles this without caching or queues.

## Contents

|    |                                    |                                                    |
|----|------------------------------------|----------------------------------------------------|
| 2  | <b>System context</b>              | Who uses the platform and what it talks to         |
| 3  | <b>Platform architecture</b>       | The main drawing: layers, all 15 modules, stores   |
| 4  | <b>Module catalogue</b>            | Each module’s job, operations, data, emails, audit |
| 5  | <b>One request, end to end</b>     | Two traced requests and the transaction rules      |
| 6  | <b>One award cycle</b>             | Every role and module from setup to results        |
| 7  | <b>Data architecture</b>           | Entities grouped by module, keyed to the cycle     |
| 8  | <b>The four rules</b>              | Where each rule is enforced and tested             |
| 9  | <b>Environments and delivery</b>   | Local, CI, production, and the file path           |
| 10 | <b>Gaps that change the design</b> | What the spec leaves open, with defaults           |
| 11 | <b>Smaller gaps and decisions</b>  | Details to settle before the schema is written     |

## How to read the drawings

●

 People and access

●

 Award configuration

●

 Intake and masking

●

 Jury, judging and approval

●

 Results and reporting

●

 Platform services

R1

 blind judging hides who applied

R2

 no assignment with a recorded conflict

R3

 every score change after submit has a reason

R4

 old applications open with their own form version

—

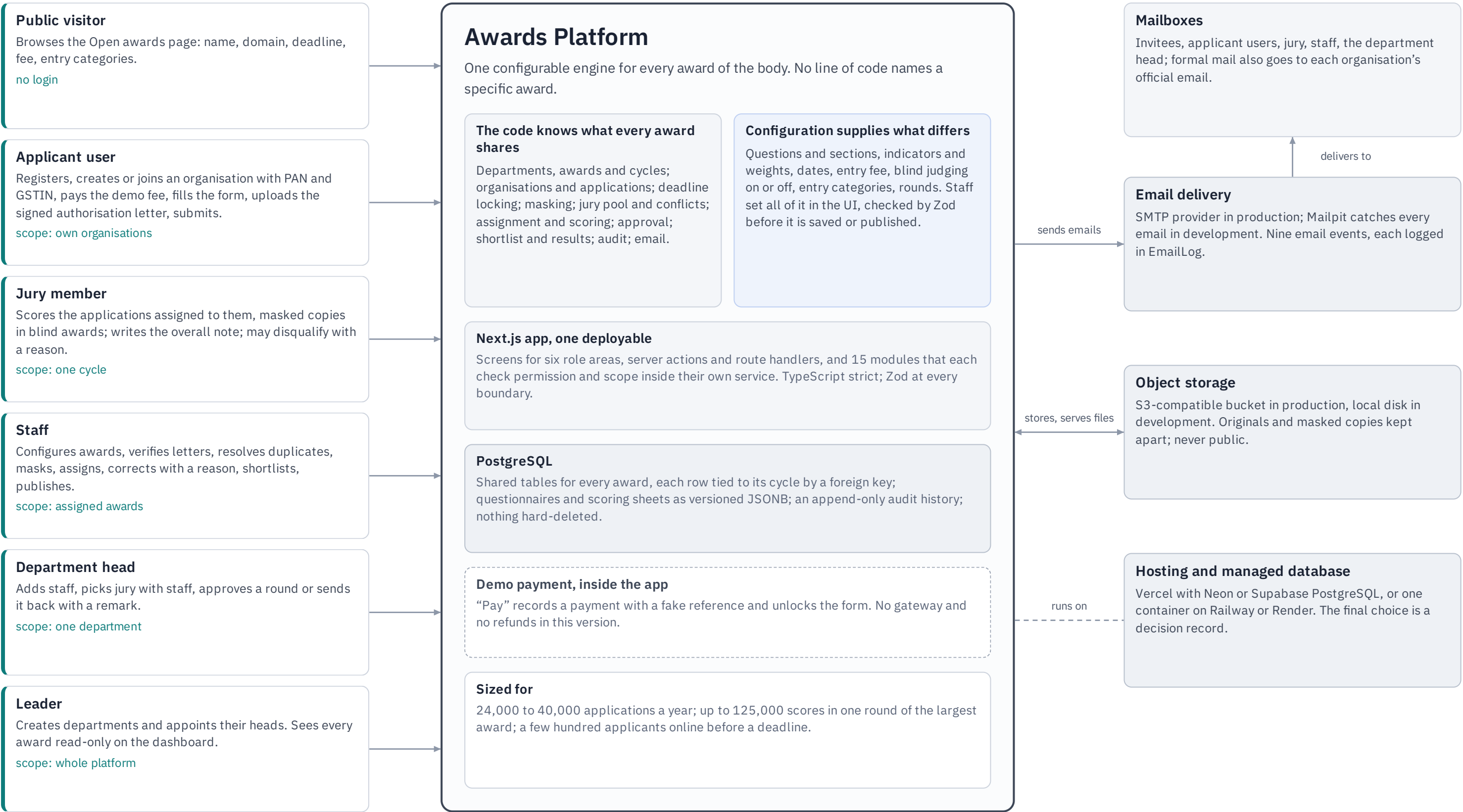
 call or data flow

- - - -

 designed for, not built yet

# Six kinds of user, one platform, and what sits outside it

The highest-level view. Everything inside the dark border is one deployable Next.js app and its database. Outside it, the platform only sends email and stores files; payments are a demo screen with no external call.



**Deliberately not connected in this version**

**Real payment gateway and refunds.** The demo sits behind one function.

**Virus scanning of uploads.** Type and size checks only.

**Member portal or single sign-on.** Email and password for everyone.

**Import of past cycles.** History starts with the first cycle run here.

**Automatic name removal from documents.** Masking is a staff task.

**Mobile app, languages, feedback reports.** Out of scope.

# Every request crosses one access pattern before any of the 15 modules touches data

One Next.js deployable on one PostgreSQL database. Pages and routes stay thin; each module owns its service, access checks, views and tests; three outside stores sit underneath. Red badges mark where the four rules of the brief are enforced.



## Build and run

### Local

Docker Compose runs PostgreSQL and Mailpit; next dev; files on local disk; the seed creates 2 awards and about 20 applications.

### CI on every pull request

GitHub Actions: lint, type check, Vitest against a real PostgreSQL service, Playwright journeys. Page 9 has the detail.

## Room kept for later

### Live round

Round type LIVE: several judges score one shortlisted application in a live room with a scoreboard. Same tables; new screens and a realtime channel.

### Row-level security

PostgreSQL policies as a second safety net under the service checks.

### Organisation tier

Above awards, if other bodies ever use the platform: true multi-tenancy.

### Payment gateway

Replaces the demo behind the same payment function.

### Email outbox

Sends after commit, in batches, if request-time email gets slow.

- People and access
- Award configuration
- Intake and masking
- Jury, judging and approval
- Results and reporting
- Platform services

- R1** blind judging
- R2** conflict of interest
- R3** score audit
- R4** question versioning

- call or data flow
- - - - designed for, not built yet

# Fifteen modules, each owning its service, access checks, views and tests

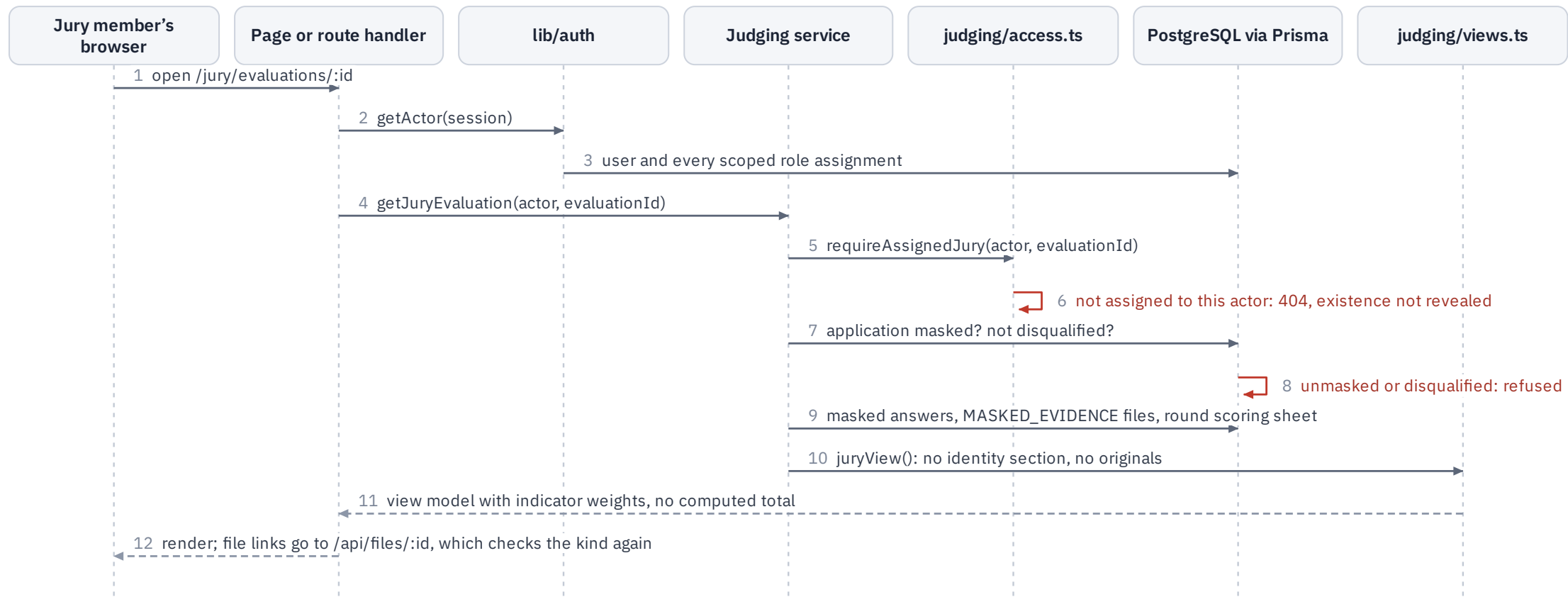
Every module folder holds service.ts, access.ts, schemas.ts, views.ts and its tests. The spec’s folder list names 13; section 11 also gives Departments and Jury pool their own operations, so both are drawn as modules here.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>Identity</div><div>src/modules/identity</div><div>Accounts, Auth.js email and password login, invite links, password resets, and the scoped role assignments every other module checks.</div><div>Operations. register, login, inviteUser, resetPassword, assignRole, getActor</div><div>Owns. User, RoleAssignment. Missing from the spec: a table for invite and reset tokens (page 10).</div><div>Called by. public (register, login); leader, dept head, staff (invites); every module (actor)</div><div>Emails. account invite, password reset</div></div>                                | <div><div>Departments</div><div>src/modules/departments</div><div>The organising body’s structure: departments, their head, staff of a department, staff of each award.</div><div>Operations. createDepartment, appointDepartmentHead (leader); addStaffToDepartment, assignStaffToAward (dept head)</div><div>Owns. Department. Writes DEPT_HEAD, DEPT_STAFF and AWARD_STAFF role rows through Identity.</div><div>Note. has operations in section 11 but no folder in section 16; drawn as its own module</div><div>Emails. invites, sent through Identity</div></div>                                                   | <div><div>Organisations</div><div>src/modules/organisations</div><div>The applying legal body: one record per PAN, shared by its users and reused across every award.</div><div>Operations. createOrganisation, joinOrganisation, getOrganisation</div><div>Owns. Organisation, OrganisationMember</div><div>Checks. PAN format and uniqueness; GSTIN format; GSTIN characters 3 to 12 equal the PAN; joining needs both</div><div>Privacy. PAN, GSTIN, address and contacts only to the organisation’s users and the award’s staff, dept head, leader</div></div>                     | <div><div>Awards and cycles</div><div>src/modules/awards</div><div>Creates awards inside a department and configures each yearly cycle. Owns cycle status and the publish gate.</div><div>Operations. createAward, createCycle, updateCycleSettings, publishCycle, extendDeadline</div><div>Owns. Award, Cycle, EntryCategory, Round</div><div>Publish gate. a section and question exist; every indicator points to a real question; weights total 100; deadline after opening; a category exists</div><div>Emails, audit. deadline extended; deadline changes</div></div>                | <div><div>Forms</div><div>src/modules/forms</div><div>R4</div><div>The questionnaire builder works on a draft; every publish freezes an immutable version with a change summary.</div><div>Operations. editDraftForm, publishFormVersion, getFormVersion, diffVersions</div><div>Owns. FormVersion (JSONB, insert-only), Cycle.draftFormSchema</div><div>Rules. inside a cycle, add sections, questions and options only; never remove or change a type; keys never reused</div><div>Emails, audit. questionnaire updated; version published. Sets Update requested on submitted applications</div></div>                                                       |
| <div><div>Scoring sheets</div><div>src/modules/scoring</div><div>Jury-only indicators attached to questions, two-level weights, and the score formula as a pure function.</div><div>Operations. editScoringSheet, validateWeights, computeScore</div><div>Owns. ScoringSheet (JSONB, one per round, lockedAt)</div><div>Formula. <math>Score = 100 \times \sum \text{section weight} \times \sum \text{indicator weight} \times \text{value}</math>; value is <math>score \div 10</math>, or Yes 1 and No 0; 2 decimals</div><div>Lock. editable until the first score in the round is saved</div></div> | <div><div>Applications</div><div>src/modules/applications</div><div>R4</div><div>The applicant’s journey up to the deadline, and the lock that ends it.</div><div>Operations. startApplication, payFee, saveAnswers, uploadFile, submit, withdraw, resolveDuplicate</div><div>Owns. Application, Answer, FileAsset (EVIDENCE, AUTH_LETTER), Payment</div><div>Rules. fee gate; one active application per organisation per cycle, else both flagged; every write compares clock.now() with the deadline; form version pinned at lock</div><div>Emails, audit. submitted; withdrawals and duplicate resolutions</div></div> | <div><div>Masking</div><div>src/modules/masking</div><div>R1</div><div>Runs only when the cycle has blind judging: staff build a jury-safe copy of each answer and file.</div><div>Operations. getMaskingWorkspace, saveMaskedAnswer, uploadMaskedFile, markMaskingDone, reopenMasking</div><div>Owns. MaskedAnswer, FileAsset (MASKED_EVIDENCE)</div><div>Rules. one application at a time, so judging can start early; reopen only while its evaluation is unsubmitted</div><div>Transaction, audit. masking done with the application status; masking done and reopened</div></div> | <div><div>Jury pool and conflicts</div><div>src/modules/jury-pool</div><div>R2</div><div>Who may judge a cycle, and which jury and organisation pairs may never meet.</div><div>Operations. addPoolMember, removePoolMember, recordConflict</div><div>Owns. Conflict (applies across all awards). JURY role rows scoped to a cycle, through Identity.</div><div>Rules. remove a member only with no submitted evaluation; a late conflict removes an unsubmitted assignment, or flags a submitted one</div><div>Who. staff and dept head. Audit: conflicts recorded</div></div>            | <div><div>Judging</div><div>src/modules/judging</div><div>R1R2R3</div><div>Assignment and round 1 scoring, with corrections and disqualification under audit.</div><div>Operations. assign, reassign, reopenEvaluation, editScoreWithReason, reinstate (staff); saveScores, submitEvaluation (jury); disqualify (both)</div><div>Owns. Evaluation, IndicatorScore, QuestionComment, DisqualificationEvent</div><div>Rules. exactly one jury per application; conflicted pairs refused; juryView from masked data only; changes after submit need a reason</div><div>Emails, audit. jury assigned; assignments, score changes, disqualify, reinstate</div></div> |
| <div><div>Approval</div><div>src/modules/approval</div><div>The department head’s decision on a whole round of a cycle.</div><div>Operations. sendForApproval (staff); approve, sendBack (dept head)</div><div>Owns. ApprovalRequest, ApprovalRemark; Round.status</div><div>Rules. send only when every non-disqualified application has a submitted evaluation; approve locks scores forever; send back needs a remark; leader refused</div><div>Transaction, emails. decision with round status; sent for approval, sent back</div></div>                                                             | <div><div>Results</div><div>src/modules/results</div><div>Turns an approved round into published outcomes for applicants.</div><div>Operations. buildShortlist, publishResults</div><div>Owns. Application.result; cycle status Results published</div><div>Rules. nothing shortlisted before approval; per category or overall; top N, cut-off or manual; ties share a rank</div><div>Emails, audit. results to applicant and official email; results published. Feeds the future live round</div></div>                                                                                                                  | <div><div>Reporting</div><div>src/modules/reporting</div><div>Read-only views across modules for the leader and department heads.</div><div>Operations. leaderDashboard, departmentDashboard, cycleSummary, awardReadOnlyView</div><div>Owns. No tables of its own. Reads through other modules’ read functions.</div><div>Shows. applications by status, masking and judging progress, pending approvals, upcoming deadlines, drill-down to a cycle</div><div>Scope. leader sees all departments; a dept head sees their own</div></div>                                              | <div><div>Audit</div><div>src/modules/audit</div><div>R3</div><div>Append-only history of every decision that matters, written inside the caller’s transaction.</div><div>Operations. record (internal, takes the transaction), listHistory</div><div>Owns. AuditEvent: insert-only, indexed by cycle, entity type and entity id</div><div>Events. score changes, disqualify and reinstate, approvals, masking, deadline changes, form versions, assignments, conflicts, duplicates, withdrawals, results</div><div>Readers. staff, dept head, leader on each application page</div></div> | <div><div>Notifications</div><div>src/modules/notifications</div><div>Every email the platform sends, and the log of it.</div><div>Operations. send(template, recipients, payload), internal only</div><div>Owns. EmailLog. Uses lib/mailer: Mailpit in development, SMTP in production.</div><div>Templates. invite, password reset, submitted, questionnaire updated, deadline extended, jury assigned, sent for approval, sent back, results published</div><div>Sent. during the request in this version</div></div>                                                                                                                                        |

# One request, end to end: who checks what, and in which order

Two requests from section 8 of the spec, traced through the layers. Route handlers never query the database; the service checks scope, state and the clock, then returns a view model built for the actor’s role.

## A jury member opens an application in a blind award



### Typed errors become responses

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| ValidationError | <b>400</b> input failed its Zod schema                     |
| ForbiddenError  | <b>403</b> role or scope does not allow it                 |
| NotFoundError   | <b>404</b> missing, or not yours to know about             |
| StateError      | <b>409</b> wrong status, past the deadline, approved round |

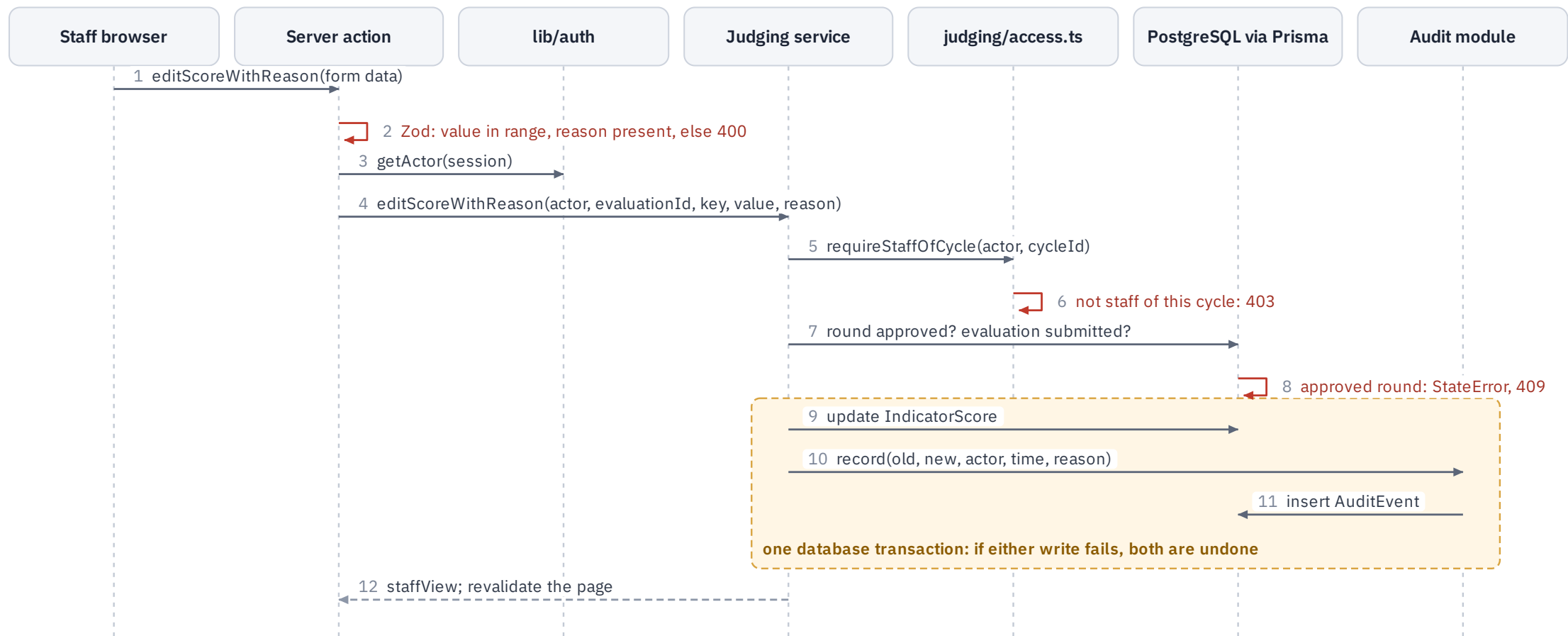
Services know nothing about HTTP; route handlers know nothing about business rules.

### Writes that must share one transaction

- A score change and its audit event
- An approval decision and the round status
- Masking done and the application status
- Resolving a duplicate: one kept, one Rejected as duplicate, and the audit event

Audit is called with the caller’s transaction, so a failed audit write rolls back the change it describes.

## Staff correct a score after the jury submitted



### Time comes from one place

Every deadline check calls `clock.now()`. Locking happens on each write, so no scheduled job is needed; tests move the clock past a deadline. Times are stored in UTC and shown in India time.

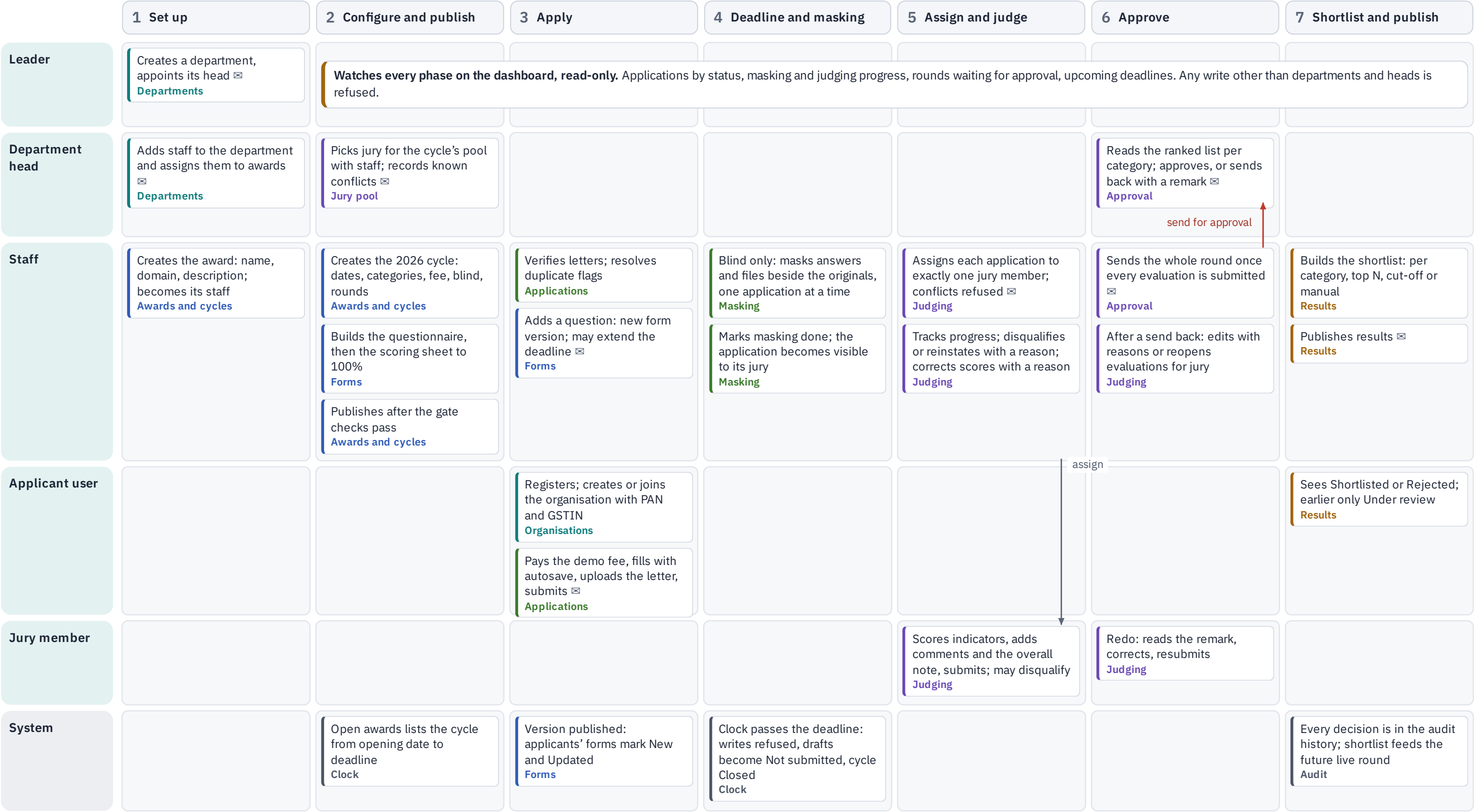
### Files take a separate, checked path

Every download goes through `/api/files/:id`, which checks the actor may read that application and that file kind. In a blind award a jury member only ever gets MASKED\_EVIDENCE; EVIDENCE and AUTH\_LETTER are refused.

On Vercel a function cannot carry a 10 MB file in either direction. Page 9 shows the signed-URL path I suggest.

# One award cycle, from setup to published results, across every role and module

Seven phases in order. Each step names the module that owns it; ✉ marks an email. Blind-only steps are skipped when the cycle has blind judging off, and nothing in the flow names a specific award.



Statues that move along this flow

- Cycle

Draft → Published → Closed at the deadline → Results published
- Round

Not started → Judging → Pending approval → Sent back, and back to Pending approval → Approved, locked forever
- Evaluation

Assigned → In progress → Submitted → Redo after a send back → Submitted
- Applicant sees

Pending → Submitted → Under review → Shortlisted or Rejected; Withdrawn and Not submitted are kept

# All awards share one set of tables; every award row is tied to its cycle

Entities from section 10, grouped by the module that writes them. Lines show the ownership chain down from the cycle, with the forked end on the many side. What varies by award lives in two JSONB columns; everything else is relational with foreign keys.

## People and organisations

**User**  
id, email (unique), name, passwordHash

**RoleAssignment**  
userId, role, departmentId?, awardId?, cycleId?  
LEADER has no scope; DEPT\_HEAD and DEPT\_STAFF a department; AWARD\_STAFF an award; JURY a cycle

**Organisation**  
pan (unique), gstin, legalName, address, officialEmail  
GSTIN characters 3 to 12 equal PAN

**OrganisationMember**  
organisationId, userId (unique pair)

**Conflict**  
juryUserId, organisationId (unique pair)  
applies across all awards; blocks assignment

## Platform

**AuditEvent**  
actorId, action, entityType, entityId, cycleId, before, after, reason  
insert only; indexed by cycle, entity type, entity id

**EmailLog**  
to, template, payload, status

**AuthToken (proposed)**  
email, type, tokenHash, expiresAt, usedAt  
invites and password resets; not in the spec's model

**Department**  
id, name (unique)

**Award**  
departmentId, name, domain, description

**Cycle**  
awardId, label, opensAt, deadlineAt, feePaise, blindJudging, status, draftFormSchema  
unique (awardId, label). Behaviour comes from these settings, never from an award's name

**Two places hold award-specific shape**  
**FormVersion.schema:** sections and questions with stable keys, types, help text, required, file rules, and a New, Updated or Unchanged marker computed at publish.  
**ScoringSheet.schema:** section weights and indicators (key, question key, type, weight). Both are checked by Zod before saving.

**Scope keys**  
Every award row reaches its cycle by a foreign key, so the database refuses mixed or orphan rows. Services filter every query by the cycles and awards in the actor's role assignments.  
  
Section 10 lists about 25 entities; these are all of them, plus one proposed token table.

**EntryCategory**  
cycleId, name (unique per cycle)

**FormVersion**  
cycleId, version, schema JSONB, changeSummary, publishedAt  
never updated after insert

**Round**  
cycleId, number, type, status  
type DOCUMENT\_REVIEW or LIVE (future)

**Application**  
cycleId, organisationId, categoryId, formVersionId, status, maskingStatus, result, duplicateFlag, updateRequested, submittedAt, lockedAt, withdrawnAt  
one active per organisation per cycle, checked in the service

**Who is the applicant?**  
The application belongs to the organisation, not to the person who filled it. The identity section (name, PAN, GSTIN, address, official email, phone) is filled from Organisation and is not part of the questionnaire schema, so it never reaches jury.  
  
Open point: if the organisation edits its profile next year, last year's application would show the new values unless identity is copied onto the application at submit (page 10).

**ScoringSheet**  
roundId, schema JSONB, lockedAt  
locked after the first saved score

**ApprovalRequest**  
roundId, decision, remark, submitted and decided by and at  
one row per submission

**Evaluation**  
roundId, applicationId, juryUserId, status, overallNote  
one jury per application in round 1

**Answer**  
applicationId, questionKey, value JSONB  
unique per application and question key

**MaskedAnswer**  
applicationId, questionKey, value, maskedBy  
jury in blind awards read only this

**FileAsset**  
applicationId, questionKey?, kind, maskedFromId?, storageKey  
EVIDENCE, AUTH\_LETTER, MASKED\_EVIDENCE

**Payment**  
applicationId, amountPaise, status, reference  
demo only

**DisqualificationEvent**  
applicationId, action, byUserId, byRole, reason  
current state is the latest event

**Where status is stored**  
Cycle.status, Round.status, Evaluation.status, plus Application.status, maskingStatus and result. Statuses that change at the deadline need care, because no job runs then (page 10).

**ApprovalRemark**  
approvalRequestId, applicationId, remark  
optional remark on one application

**IndicatorScore**  
evaluationId, indicatorKey, value  
0 to 10, or 1 and 0 for Yes and No

**QuestionComment**  
evaluationId, questionKey, comment

**Storage, outside the database**  
FileAsset holds a storageKey, never the bytes. Originals and masked copies live under separate key prefixes in a private bucket. maskedFromId links a masked file to its original. Every download goes through the checked file route.  
Files: PDF, JPG, PNG, DOCX, XLSX; 10 MB each.

**Conventions on every table**

- Money as integer paise; times in UTC, shown in Asia/Kolkata
- Nothing an applicant, jury member or staff member produced is hard-deleted; disqualified, withdrawn and duplicate records stay
- Answers are keyed by questionKey, so they carry across versions; an application pins formVersionId at the deadline
- Evaluation is unique per round, application and jury, so live rounds can add judges without a model change
- A new award is only new rows; no table is ever named after an award

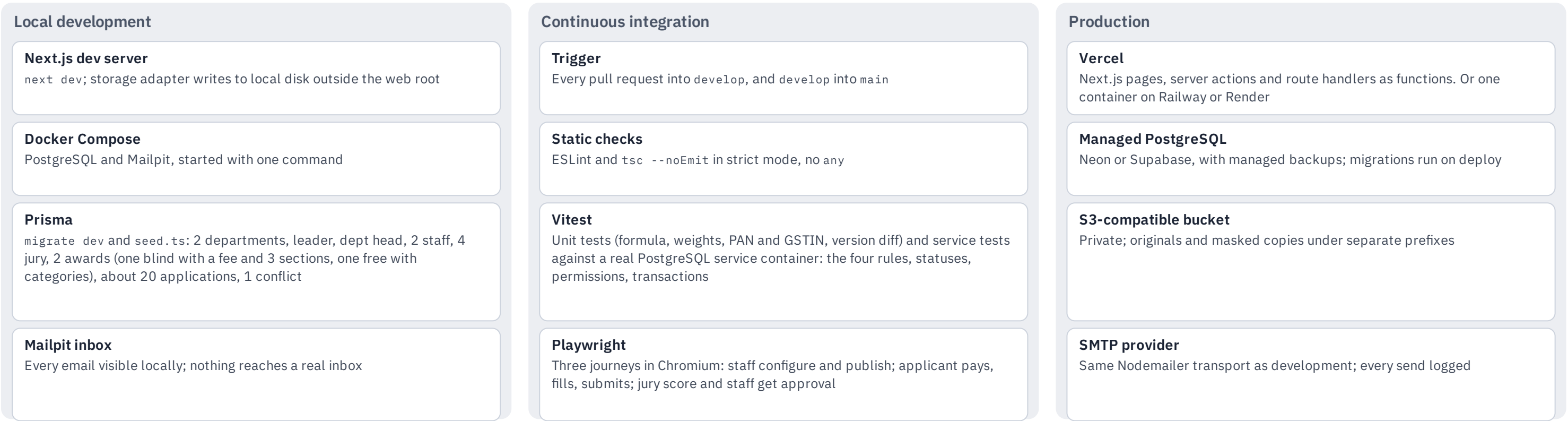
# Each of the four rules is enforced on the server, in more than one layer

The UI only mirrors a rule; the service is where it is decided, and the database or storage adds a second guard where it can. Every row ends with the tests that prove it and the limit the spec states openly.

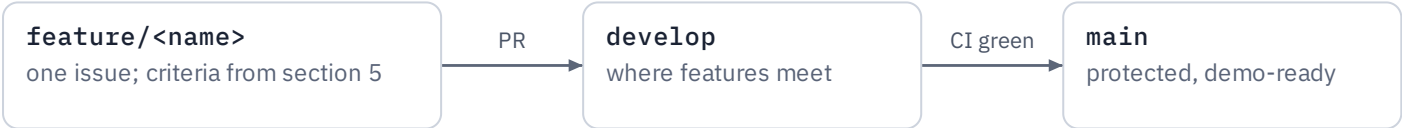
|                                                                                                                | Screens and routes                                                                                                                          | Service and access.ts                                                                                                                       | View model                                                                              | Database                                                                                                                                    | Files and storage                                                                          | Tests that prove it                                                                                                                                                   | Known limit                                                                            |
|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <div>R1</div> <div>Blind judging hides who applied</div> <div>Modules: Masking, Judging</div>                  | Jury pages call jury services only; middleware keeps jury out of staff screens                                                              | requireAssignedJury; refuse an unmasked or disqualified application; behaviour read from cycle.blindJudging                                 | juryView built only from MaskedAnswer and masked files; identity section never included | Originals in Answer, masked copies in MaskedAnswer; FileAsset.kind separates file copies                                                    | /api/files/:id refuses EVIDENCE and AUTH_LETTER to jury in blind awards; bucket is private | No identity field in any jury response; original download refused; unmasked application refused; staff see both copies; non-blind jury see originals                  | Masking quality depends on staff; a name left inside a document cannot be detected     |
| <div>R2</div> <div>No assignment with a recorded conflict</div> <div>Modules: Jury pool, Judging</div>         | Assignment list hides a conflicted jury member                                                                                              | assign and reassign refuse a conflicted pair, even from a direct API call; a late conflict removes an unsubmitted assignment                | Staff view of the pool shows recorded conflicts                                         | Conflict unique per jury user and organisation; applies across all awards                                                                   | Not involved                                                                               | Assigning a recorded conflict through the service is refused; a conflict recorded later removes the unsubmitted assignment                                            | A conflict nobody recorded cannot be caught; the client says staff hold this knowledge |
| <div>R3</div> <div>Who changed a score, and why</div> <div>Modules: Judging, Audit</div>                       | A reason field is required when editing after submission                                                                                    | After submit every change needs a reason; jury may edit only in Redo; an approved round refuses all changes                                 | Application page shows history to staff, dept head and leader                           | Score update and AuditEvent insert in one transaction; AuditEvent insert-only (a trigger would make that a database fact, page 11)          | Not involved                                                                               | Change without a reason refused; old value, new value, actor, time and reason written; failed audit write rolls back the score; change after approval refused         | Draft scores before the first submission are not audited, by design                    |
| <div>R4</div> <div>Last year’s applications still read correctly</div> <div>Modules: Forms, Applications</div> | Form renderer always loads the application’s own version; New and Updated markers come from the diff                                        | Published versions are immutable; inside a cycle, no removing questions or changing types; options add-only; version pinned at the deadline | Applicant and staff views render with the pinned FormVersion                            | FormVersion insert-only and unique per cycle and version; Answer keyed by questionKey; Application.formVersionId                            | Not involved                                                                               | Editing a published version refused; removing a question in a cycle refused; a 2025 application opens with its 2025 version; answers carry over                       | Removing a question still needs a new cycle, by design                                 |
| <div>Unit, Vitest</div> <div>Score formula, weight validation, PAN and GSTIN checks, version diff</div>        | <div>Service, Vitest on a real PostgreSQL</div> <div>The four rules, status changes, permissions per role, transactions and rollbacks</div> |                                                                                                                                             |                                                                                         | <div>End to end, Playwright</div> <div>Staff configure and publish; applicant pays, fills, submits; jury score and staff get approval</div> |                                                                                            | <div>Not checked by tests</div> <div>Masking quality, real inbox delivery, load, browsers other than Chromium, security beyond permission checks, file contents</div> |                                                                                        |

# Three environments, one codebase, and a file path that fits the hosting limits

Local and CI run the same PostgreSQL and mail catcher through Docker. Production swaps disk for a bucket and Mailpit for SMTP behind the same lib interfaces, so no module changes between environments.



## Branches and the definition of done



**Pull request checklist**

☐ Acceptance criteria of the issue met

☐ Tests added, including any rule touched

☐ Lint, type check and tests pass in CI

☐ Docs and decision records updated

☐ No code branches on a specific award

☐ Every new service function checks permission and scope

☐ Responses use a view model, never raw rows

☐ Diff reviewed line by line

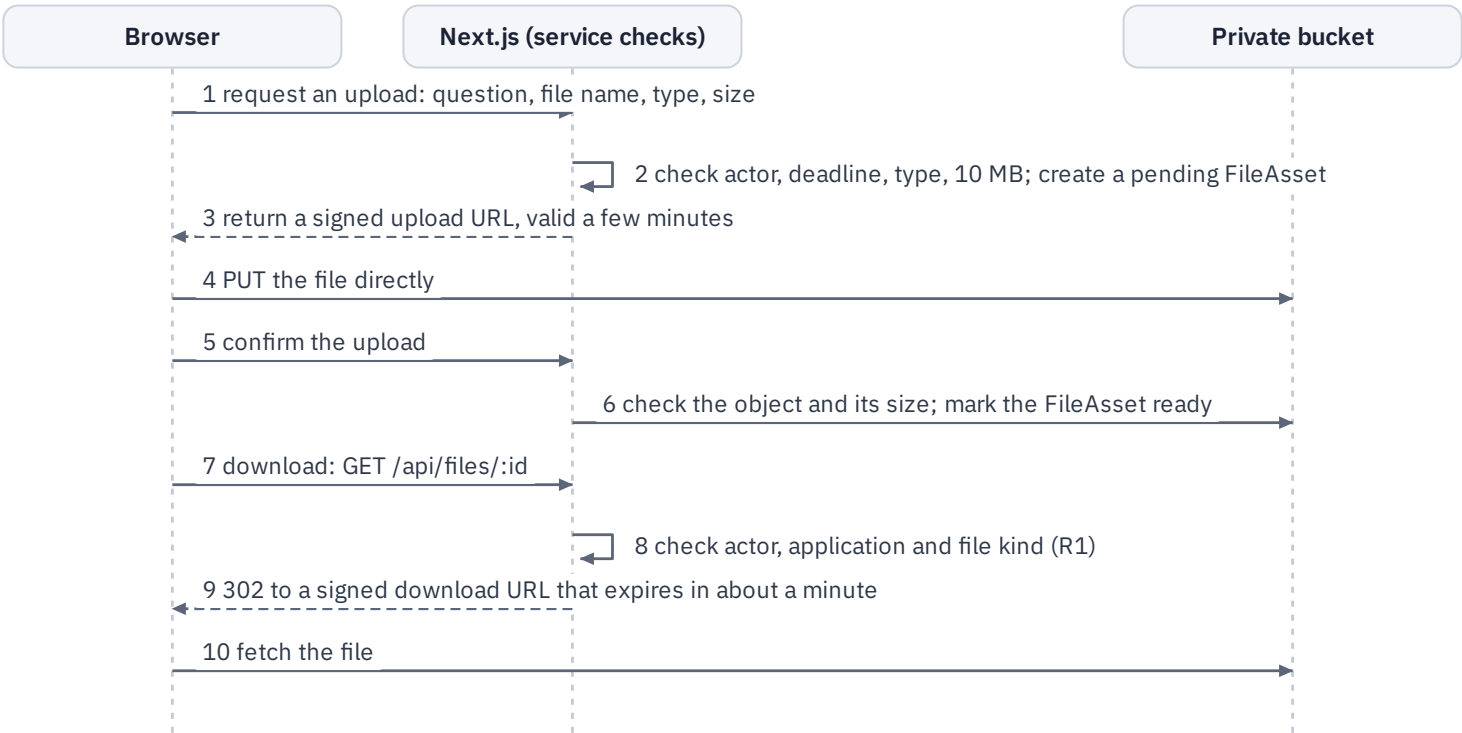
Commits follow Conventional Commits. Daily update: Done, Next, Stuck, and Plan changed when it did.

**Configuration and secrets**

Secrets only in environment variables, validated by a Zod schema at start-up so a missing value fails fast; a committed .env.example lists them. Suggested names: DATABASE\_URL, AUTH\_SECRET, APP\_URL, SMTP\_HOST, SMTP\_PORT, SMTP\_USER, SMTP\_PASS, MAIL\_FROM, STORAGE\_DRIVER (disk or s3), S3\_BUCKET, S3\_REGION, S3\_ENDPOINT, S3\_ACCESS\_KEY, S3\_SECRET\_KEY.

## Proposed file path for production (not in the spec)

Vercel functions cap request and response bodies at 4.5 MB, and Next.js server actions default to a 1 MB body. The spec allows 10 MB files and serves them through a checked route, so on Vercel the bytes should travel straight between the browser and the bucket, with the server deciding only who may do it.



On a container host (Railway or Render) the same route can stream files directly, and the storage interface stays the same either way. The choice belongs in the hosting decision record.

# Gaps that change the architecture, each with the default I would build

None of these are among the 16 questions already in section 18. Each card says what the spec states, the default to build if nobody decides, and the decision needed where there is one.

## 1 The module list disagrees with itself

The drawing in section 8 shows 12 modules, the folder list in section 16 names 13, and the operations table in section 11 implies 15. Departments and Jury pool have operations but no folder, and Reporting is missing from the old drawing.

**Default until decided.** Fifteen module folders, adding `departments` and `jury-pool`; page 3 replaces the section 8 drawing.

**Decide.** Add those two folders, or fold Departments into Identity and Jury pool into Judging?

**Affects:** folder structure, pages 3 and 4

## 2 10 MB files will not pass through a Vercel function

Uploads go through the app and every download through one checked route. Vercel caps function request and response bodies at 4.5 MB, and Next.js server actions accept 1 MB by default.

**Default until decided.** Signed direct uploads and short-lived signed download URLs, behind the same access checks (page 9). Or host one container on Railway or Render, where streaming works.

**Decide.** Which host: Vercel, or a container?

**Affects:** Applications, Masking, lib/storage, the file route

## 3 Emails are sent while the request is handled

Publishing a new form version or the results sends one email per applicant plus one to each organisation’s official address: up to about 1,000 for the largest award in a single request. A function can time out, and an email can leave for a change whose transaction later rolls back.

**Default until decided.** Use EmailLog as an outbox: insert rows marked Pending in the same transaction, then send in small batches after the response or from a cron route.

**Decide.** Add the outbox now, or keep request-time sending with a batch cap?

**Affects:** Notifications, Forms, Results, EmailLog

## 4 Some statuses change with time, and no job changes them

The cycle becomes Closed and unsubmitted drafts become Not submitted “automatically at the deadline”, but there are no background jobs.

**Default until decided.** Derive these from `clock.now()` whenever they are read, and store them on the next write. Dashboards and filters use the derived value.

**Decide.** Is derived-on-read acceptable for the leader’s dashboard counts?

**Affects:** Awards and cycles, Applications, Reporting

## 5 Invite and reset links have nowhere to live

Invite links and expiring password-reset links are required, but the data model has no table for their tokens. Auth.js with email and password keeps sessions as JWTs, not database rows.

**Default until decided.** An AuthToken table: hashed token, type, expiry, used-at, single use. The session carries only the user id; roles load from the database on every request, as the spec already says.

**Affects:** Identity, data model (page 7)

## 6 Removing an assignment conflicts with never hard-deleting

Reassigning, or recording a conflict after assignment, removes the evaluation. Evaluation has no status for that, and its unique key includes the jury member.

**Default until decided.** A Revoked status with a reason, and a partial unique index that allows one active evaluation per application per round. Draft scores stay on record.

**Affects:** Judging, Jury pool, Evaluation

## 7 The identity section reads the live organisation profile

Identity fields are filled from Organisation. If the organisation changes its name or address next year, its 2026 application would show the new values.

**Default until decided.** Copy the identity fields onto the application at submit, refresh them until the deadline, and freeze them at the lock. Rule 4’s promise then covers identity too.

**Decide.** Snapshot identity onto the application like this?

**Affects:** Applications, Organisations

## 8 Verifying authorisation letters has no operation or outcome

The permission matrix gives staff “verify authorisation letters”, but there is no service operation, no field for the result, and no rule for a rejected letter.

**Default until decided.** `verifyAuthLetter` with a letter status (Pending, Verified, Rejected with a reason), audited. Staff ask for a new letter before the deadline.

**Decide.** Does an unverified letter block masking and assignment, or only show a warning?

**Affects:** Applications, Audit, staff application screen

# Smaller gaps, and every decision in one place

Details that do not move a box on page 3 but should be settled before the Prisma schema is written on day 3.

## 9 The pre-filled letter template needs a generator

The template is downloaded “pre-filled with organisation and award names”. Nothing in the stack produces a document.

**Default until decided.** Generate it on the server inside Applications from one platform default template.

**Decide.** PDF or DOCX? Organisations may want to print it on their letterhead, which favours DOCX.

**Affects:** Applications

## 11 Unique keys with empty scope columns

RoleAssignment must be unique per user, role and scope, and there is exactly one leader. PostgreSQL treats empty (null) columns as distinct in a normal unique index, so duplicates slip in.

**Default until decided.** NULLS NOT DISTINCT on the index (PostgreSQL 15 and later) or partial unique indexes, plus a partial unique index on the LEADER role.

**Affects:** Identity, migrations

## 13 “Safe as is” files need a representation

Staff may mark a file safe instead of uploading a masked copy; the model does not say how that is stored.

**Default until decided.** A MASKED\_EVIDENCE row pointing at the same storage key, so the jury file check stays a single rule.

**Affects:** Masking, FileAsset

## 15 The live round will need a realtime channel

The live room updates a scoreboard while several judges score. Serverless functions do not hold WebSocket connections open.

**Default until decided.** When it is built: short polling first, or a hosted realtime service. Noted now so it is not a surprise.

**Affects:** future Live round

## 10 Append-only audit is only a convention

AuditEvent is insert-only because application code promises it. Role, department and letter changes are not in the list of audited events.

**Default until decided.** A database trigger that rejects UPDATE and DELETE on AuditEvent; add role assignments, staff changes and letter verification to the audited events.

**Affects:** Audit, migrations

## 12 No throttling on login, no monitoring in production

Nothing limits repeated login or reset attempts, and no error tracking or structured logging is named.

**Default until decided.** A simple limit per email and per IP on login and reset; structured server logs and an error tracker in production.

**Affects:** Identity, deployment

## 14 Totals: computed or stored?

The ranked list needs totals for up to 500 applications with 250 indicators each. Where the total lives is not specified.

**Default until decided.** Compute with the pure computeScore on read; snapshot totals and ranks when the round is approved, so results never shift.

**Affects:** Scoring, Approval, Results

## Decisions needed

Numbers match the cards. Until a decision is made, the default on each card is what gets built.

- 1 Add those two folders, or fold Departments into Identity and Jury pool into Judging?
- 2 Which host: Vercel, or a container?
- 3 Add the outbox now, or keep request-time sending with a batch cap?
- 4 Is derived-on-read acceptable for the leader’s dashboard counts?
- 7 Snapshot identity onto the application like this?
- 8 Does an unverified letter block masking and assignment, or only show a warning?
- 9 PDF or DOCX? Organisations may want to print it on their letterhead, which favours DOCX.

## Still open from section 18

The drawings follow the spec’s own defaults for its 16 questions. Three of them would move a box or a line if answered differently:

- **Q2** If the leader must also sign off, Approval gains a second decision and the leader stops being read-only.
- **Q4, Q5** If department heads create awards or edit scores, their steps on page 6 move into Awards and Judging.
- **Q1** If plants apply separately, Organisation needs a unit below it, and the duplicate rule changes its key.